

RELATÓRIO DE AUDITORIA DE SEGURANÇA

Projeto Jeffrey — Análise de Segurança Completa

Versão 1.0 — Junho 2025

Classificação: CONFIDENCIAL

Stack Detectada	Python 3.14 · FastAPI · LangGraph · PostgreSQL 16 + pgvector
Achados	9 (2 críticos · 3 altos · 3 médios · 1 baixo)
Pontos Fortes	13 controles de segurança validados
Recomendações	7 ações priorizadas (P1–P7)

SUMÁRIO

1. Visão Geral da Auditoria
2. Stack Tecnológica Detectada
3. Resumo Executivo
4. Distribuição dos Achados
5. Achados Detalhados
 - 5.1 Banco sem Tranca (3 achados)
 - 5.2 Permissão no Navegador (1 achado)
 - 5.3 IDOR (1 achado)
 - 5.4 Chaves Expostas (3 achados)
 - 5.5 Inputs sem Tratamento (1 achado)
6. Pontos Fortes e Controles Válidos
7. Recomendações Priorizadas
8. Issues para GitHub
9. Conclusão

1. VISÃO GERAL DA AUDITORIA

Este relatório apresenta os resultados da auditoria de segurança realizada no Projeto Jeffrey, um sistema de agente de IA baseado em LangGraph com interface CLI. A auditoria avaliou a segurança da aplicação em múltiplas dimensões: isolamento de dados, autorização, autenticação, gerenciamento de segredos e validação de entrada.

A análise identificou **9 achados** distribuídos em 5 categorias, incluindo 2 de severidade crítica que requerem ação imediata. Também foram validados **13 pontos fortes** que demonstram uma base sólida de controles de segurança na aplicação.

2. STACK TECNOLÓGICA DETECTADA

Camada	Tecnologia	Detalhes
Linguagem	Python 3.14	Runtime principal
Framework	FastAPI + Starlette + LangGraph	API na porta 8000; approvals no Starlette; loop do agente via LangGraph
ORM/Banco	SQLAlchemy + psycopg	PostgreSQL 16 + pgvector para embeddings
Autenticação	Bearer token middleware	JEFREY_API__SECRET_KEY — somente no app Starlette (approvals)
Frontend	CLI (typer + rich + httpx)	Sem frontend web
Deploy	Docker Compose	postgres, redis, n8n, mcp-server, jeffrey-api

3. RESUMO EXECUTIVO

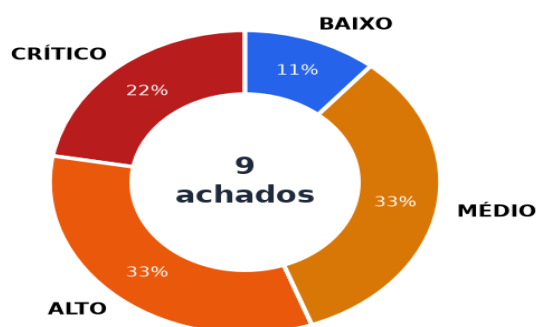
A auditoria revelou vulnerabilidades significativas em **isolamento de dados** e **autorização**. Os dois achados críticos — memória sem isolamento por usuário e chave de API real em .env — representam riscos imediatos que devem ser tratados como prioridade máxima.

Os achados de severidade alta (RBAC ausente em endpoints, IDOR em aprovações) indicam lacunas no modelo de autorização que poderiam ser explorados por qualquer usuário autenticado. As recomendações deste relatório seguem uma ordem de prioridade (P1–P7) que deve guiar o remediamento.

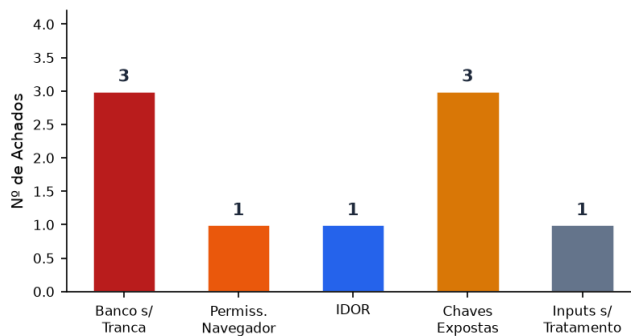
Severidade	Quantidade	Ação Necessária
CRÍTICO	2	Remediar imediatamente
ALTO	3	Remediar antes do próximo release
MÉDIO	3	Remediar no curto prazo
BAIXO	1	Melhorar na próxima iteração

4. DISTRIBUIÇÃO DOS ACHADOS

Distribuição por Severidade



Achados por Categoria



5. ACHADOS DETALHADOS

5.1 Banco sem Tranca

[1.1] Tabelas de memória sem isolamento por usuário/tenant

CRÍTICO

Categoria: Banco sem Tranca

Ref: CWE-863: Incorrect Authorization

Descrição: As 5 tabelas de memória (episódica, semântica, preferência, procedimental e operacional) NÃO possuem coluna `user_id`. Todas as consultas retornam TODOS os dados de TODOS os usuários. Um deploy multiusuário vazaria memórias entre usuários distintos.

Arquivos afetados:

- `src/jefrey/core/models.py` (todas as classes `_MemoryMixin`, linhas 17-38)
- `src/jefrey/core/pg_memory.py` (métodos `search`, `get`, `update`, `delete`, `list_recent`)

Impacto: Vazamento completo de dados entre usuários em ambiente multiusuário.

Recomendação: Adicionar coluna `user_id` a todas as tabelas de memória e filtrar todas as consultas.

[1.2] `ApprovalManager.get_pending()` retorna todas as aprovações sem filtro

ALTO

Categoria: Banco sem Tranca

Ref: CWE-863: Incorrect Authorization

Descrição: `GET /approvals/pending` retorna TODAS as aprovações pendentes sem filtragem por usuário. Qualquer usuário autenticado visualiza aprovações de outros usuários.

Arquivos afetados:

- `src/jefrey/core/hitl.py`, método `get_pending()` (linha ~100)

Impacto: Exposição de aprovações pendentes de outros usuários.

Recomendação: Filtrar aprovações por `user_id` ou thread ownership.

[1.3] Ausência de Row Level Security (RLS) no banco de dados

MÉDIO

Categoria: Banco sem Tranca

Ref: CWE-863: Incorrect Authorization

Descrição: O PostgreSQL não possui políticas RLS. Mesmo que o filtro no nível da aplicação seja adicionado depois, não existe defesa em profundidade.

Arquivos afetados:

- `docker-compose.yml`
- `src/jefrey/core/db.py`

Impacto: Falha de defesa em profundidade; um bug na aplicação expõe todos os dados.

Recomendação: Implementar RLS no PostgreSQL como camada extra de proteção.

5.2 Permissão no Navegador

[2.1] Endpoints da API (chat, memory) sem verificações RBAC

ALTO

Categoria: Permissão no Navegador

Ref: CWE-862: Missing Authorization

Descrição: O RBAC é aplicado apenas no ToolExecutor e no gateway MCP. Os endpoints FastAPI (chat, memory) aceitam requisições de qualquer pessoa sem verificar o papel. Um usuário convidado pode conversar com o agente e acessar a busca de memória.

Arquivos afetados:

- src/jeffrey/api/chat.py (POST /chat) — sem validação de papel
- src/jeffrey/api/memory.py (GET /memory/search, GET /memory) — sem validação de papel

Impacto: Usuários não autorizados acessam funcionalidades restritas.

Recomendação: Adicionar middleware RBAC em todos os endpoints FastAPI.

5.3 IDOR

[3.1] Endpoint decide de aprovação sem verificação de posse

ALTO

Categoria: IDOR

Ref: CWE-639: Authorization Bypass Through User-Controlled Key

Descrição: POST /approvals/{id}/decide requer apenas o token Bearer — qualquer usuário autenticado pode aprovar/rejeitar QUALQUER aprovação de QUALQUER thread. Não há verificação de que o aprovador tem autoridade sobre aquela aprovação ou thread específica.

Arquivos afetados:

- src/jeffrey/api/approvals.py, função decide() (linha ~65)

Impacto: Um usuário malicioso pode aprovar ou rejeitar operações de outros usuários.

Recomendação: Adicionar verificação de posse (ownership) antes de permitir decisão.

5.4 Chaves Expostas

[4.1] Chave real da API Tavily em arquivo .env

CRÍTICO

Categoria: Chaves Expostas

Ref: CWE-798: Use of Hard-coded Credentials

Descrição: JEFREY_TAVILY_API_KEY=tvly-dev-qwgiq-... está presente no diretório de trabalho. Embora .env esteja no .gitignore, se o padrão falhar ou o arquivo for commitado acidentalmente, a chave fica exposta publicamente.

Arquivos afetados:

- .env, linha 29

Impacto: Chave de API de terceiros exposta, possíveis custos financeiros e abuso.

Recomendação: Usar variáveis de ambiente ou Docker secrets; roterar a chave atual.

[4.2] Credenciais de banco codificadas no docker-compose.yml

MÉDIO

Categoria: Chaves Expostas

Ref: CWE-798: Use of Hard-coded Credentials

Descrição: POSTGRES_USER: jeffrey, POSTGRES_PASSWORD: jeffrey, POSTGRES_DB: jeffrey. Credenciais padrão são aceitáveis para desenvolvimento, mas devem usar secrets/variáveis em produção.

Arquivos afetados:

- docker-compose.yml, linhas 10-12

Impacto: Credenciais padrão facilmente adivinháveis em produção.

Recomendação: Usar Docker secrets ou variáveis externas em produção.

[4.3] Chave secreta padrão vazia permite bypass completo de autenticação

BAIXO

Categoria: Chaves Expostas

Ref: CWE-521: Weak Password Requirements

Descrição: secret_key: str = "" — Quando vazia, o middleware de auth rejeita TODAS as requisições (projetado corretamente), mas não há validação na inicialização que avisa ou recusa iniciar quando secret_key está vazia em produção.

Arquivos afetados:

- src/jefrey/core/config.py, classe APISettings (linha ~270)

Impacto: Possível confusão em produção; middleware pode rejeitar tudo ou aceitar tudo dependendo da config.

Recomendação: Adicionar validação na inicialização para secret_key em modo produção.

5.5 Inputs sem Tratamento

[5.1] Padrões regex do content_guard são limitados

MÉDIO

Categoria: Inputs sem Tratamento

Ref: CWE-20: Improper Input Validation

Descrição: Apenas 7 padrões de detecção. Faltam: prompt injection codificado, padrões multilíngues, bypass via Unicode, instruções em base64, tags de system prompt aninhadas.

Arquivos afetados:

- src/jefrey/core/content_guard.py

Impacto: Ataques de prompt injection podem bypassar as proteções atuais.

Recomendação: Expandir os padrões com listas de verificação OWASP LLM e testes adversariais.

6. PONTOS FORTES E CONTROLES VÁLIDOS

A auditoria também identificou **13 controles de segurança** que estão funcionando corretamente. Estes pontos representam uma base sólida que deve ser mantida e expandida.

#	Controle	Status
1	Autenticação Bearer token em todos os endpoints de aprovação	✓ VÁLIDO
2	Content guard aplicado à entrada do usuário em POST /chat	✓ VÁLIDO
3	Content guard aplicado à saída externa do MCP	✓ VÁLIDO
4	RBAC aplicado no ToolExecutor e gateway MCP	✓ VÁLIDO
5	Validação UUID antes de tocar no banco de dados	✓ VÁLIDO
6	arguments_json omitido da resposta de listagem	✓ VÁLIDO
7	mode='off' não contorna o RBAC	✓ VÁLIDO
8	Papel resolvido no lado do servidor	✓ VÁLIDO
9	Deteção de callable síncrono no ToolExecutor	✓ VÁLIDO
10	Fallback de auditoria em caso de falha no PostgreSQL	✓ VÁLIDO
11	ToolRegistry fail-safe: ferramentas não registradas são bloqueadas	✓ VÁLIDO
12	.env devidamente no .gitignore	✓ VÁLIDO
13	Bypass do admin funciona corretamente	✓ VÁLIDO

7. RECOMENDAÇÕES PRIORIZADAS

As seguintes recomendações estão ordenadas por prioridade de implementação. A prioridade P1 deve ser executada imediatamente.

Prioridade	Ação	Severidade
P1	Adicionar user_id às tabelas de memória + filtrar todas as consultas	CRÍTICO
P2	Adicionar middleware RBAC aos endpoints FastAPI	ALTO
P3	Adicionar verificação de posse ao endpoint decide de aprovação	ALTO
P4	Adicionar validação na inicialização para secret_key	MÉDIO
P5	Expandir padrões do content_guard	MÉDIO
P6	Usar Docker secrets para credenciais em produção	MÉDIO
P7	Considerar RLS do PostgreSQL como defesa em profundidade	BAIXO

8. ISSUES PARA GITHUB

As issues abaixo estão formatadas para criação direta no GitHub. Cada issue inclui título, labels e corpo descritivo.

ISSUE #1: [Segurança] Memória sem isolamento por usuário — vazamento de dados entre usuários

Labels: `security critical`

Descrição

As 5 tabelas de memória (episódica, semântica, preferência, procedimental e operacional) não possuem coluna ``user_id``. Todas as consultas no ``pg_memory.py`` retornam TODOS os dados de TODOS os usuários.

Arquivos afetados

- ``src/jefrey/core/models.py`` (todas as classes ``_MemoryMixin``, linhas 17-38)
- ``src/jefrey/core/pg_memory.py`` (métodos `search`, `get`, `update`, `delete`, `list_recent`)

Impacto

Um deploy multiusuário vazaria memórias entre usuários distintos.

Solução

- Adicionar coluna ``user_id`` (UUID, NOT NULL) a todas as tabelas de memória
- Filtrar todas as consultas por ``user_id``
- Adicionar migração Alembic para coluna existente
- Considerar RLS como defesa em profundidade

ISSUE #2: [Segurança] API endpoints (chat/memory) sem verificação RBAC

Labels: `security high`

Descrição

O RBAC é aplicado apenas no ``ToolExecutor`` e no gateway MCP. Os endpoints FastAPI (`chat`, `memory`) aceitam requisições de qualquer pessoa sem verificar o papel.

Arquivos afetados

- ``src/jefrey/api/chat.py`` – POST `/chat` sem validação de papel
- ``src/jefrey/api/memory.py`` – GET `/memory/search`, GET `/memory` sem validação de papel

Impacto

Um usuário convidado pode conversar com o agente e acessar a busca de memória.

Solução

Adicionar middleware ou dependência RBAC em todos os endpoints FastAPI.

ISSUE #3: [Segurança] Approval decide sem verificação de posse (IDOR)

Labels: `security high`

Descrição

POST /approvals/{id}/decide requer apenas o token Bearer – qualquer usuário autenticado pode aprovar/rejeitar QUALQUER aprovação de QUALQUER thread.

Arquivos afetados

- ``src/jefrey/api/approvals.py``, função ``decide()`` (linha ~65)

Impacto

Um usuário malicioso pode aprovar ou rejeitar operações de outros usuários.

Solução

Adicionar verificação de ownership: confirmar que o aprovador tem autoridade sobre a aprovação ou thread específica antes de processar a decisão.

ISSUE #4: [Segurança] Chaves padrão e validação de startup ausente

Labels: [security](#), [medium](#)

Descrição

Dois problemas agrupados:

1. ``secret_key`` vazio por padrão – sem validação na inicialização
2. Credenciais de banco codificadas no ``docker-compose.yml``

Arquivos afetados

- ``src/jefrey/core/config.py`` – ``secret_key: str = ""`` (linha ~270)
- ``docker-compose.yml`` – credenciais padrão (linhas 10-12)

Solução

1. Adicionar validação na inicialização que recusa iniciar em modo produção com `secret_key` vazio
2. Usar Docker secrets ou variáveis de ambiente para credenciais em produção

ISSUE #5: [Segurança] content_guard com padrões insuficientes

Labels: [security](#), [medium](#)

Descrição

O `content_guard` possui apenas 7 padrões de detecção. Faltam verificações para:

- Prompt injection codificado
- Padrões multilíngues
- Bypass via Unicode
- Instruções em base64
- Tags de system prompt aninhadas

Arquivos afetados

- ``src/jefrey/core/content_guard.py``

Solução

Expandir os padrões seguindo OWASP LLM Security Top 10 e realizar testes adversariais.

9. CONCLUSÃO

O Projeto Jeffrey demonstra uma base sólida de controles de segurança, com 13 mecanismos validados incluindo autenticação Bearer, content guard, RBAC no ToolExecutor e validação UUID. No entanto, a auditoria identificou lacunas críticas em **isolamento de dados** e **autorização** que devem ser tratadas com urgência.

A ação imediata deve focar nos **2 achados críticos** (P1 e P6) e nos **3 achados altos** (P2 e P3). A implementação das 7 recomendações priorizadas elevará significativamente o postura de segurança do sistema e o preparará para um deploy multiusuário seguro em produção.

Recomenda-se uma nova auditoria de segurança após a implementação das recomendações P1–P3 para validar as correções e identificar possíveis regressões.

Auditor Responsável

Jefrey Security Audit Team

Data: 30/08/2026

Versão 1.0